

# Graph-Guided Heterogeneous Dual-Population Evolution for Multi-Objective Flexible Job Shop Scheduling

Jie He<sup>1</sup>, Changqi Han<sup>1</sup>, Di Gu<sup>1</sup>, Yue Qi<sup>1\*</sup>, Shihong Duan<sup>1\*</sup>, Jian Liu<sup>1</sup>

<sup>1\*</sup>School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing, 100083, China.

\*Corresponding author(s). E-mail(s): [qiyuee@ustb.edu.cn](mailto:qiyuee@ustb.edu.cn); [duansh@ustb.edu.cn](mailto:duansh@ustb.edu.cn);  
Contributing authors: [hejie@ustb.edu.cn](mailto:hejie@ustb.edu.cn); [disparity\\_han@163.com](mailto:disparity_han@163.com); [gudigd@163.com](mailto:gudigd@163.com);

## Abstract

Flexible job shop scheduling (FJSP) under multiple conflicting objectives is a core decision problem in flexible manufacturing, for which multi-objective evolutionary algorithms are attractive because a single run returns a set of trade-off schedules. Their effectiveness is limited by two practical issues. Randomly or rule-initialized populations ignore instance-specific structure, and the conventional single-population design makes convergence and diversity difficult to control in the same search process. To address these issues, this paper develops a heterogeneous dual-population evolutionary optimization approach guided by an instance-specific learned prior: offline, an edge-conditioned graph neural network with global attention pooling encodes each instance as a static heterogeneous disjunctive graph and predicts a distribution over dispatching-rule combinations from multi-objective soft labels. Online, the predicted distribution initializes two populations with different search roles: a main population that allocates individuals across rule combinations in proportion to the full predicted distribution for broad coverage, and an exploratory population seeded by the single highest-probability rule combination for deep search. Bidirectional exchange and adaptive mixed evolution link the two searches. On the synthetic validation set, the learned prior alone improves the hypervolume by 123.7% over a randomly initialized NSGA-II and the complete framework adds a further 30.7% over this single-population prior. Across both the synthetic instances and the public BehnkeGeiger benchmark, the complete framework attains the best hypervolume, IGD, and front-distribution metrics—spacing  $\Delta$  and its cross-run stability  $\Delta_v$ —among the compared methods.

**Keywords:** Flexible job shop scheduling, Multi-objective optimization, Advanced manufacturing, Graph neural network, Dispatching-rule recommendation, Dual-population evolutionary algorithm

## 1 Introduction

Modern production systems increasingly rely on flexible production resources, short response cycles, and data-driven decision support. In a flexible job shop, each operation can usually be processed by one of several eligible machines, so both machine assignment and operation sequencing

must be determined. The flexible job shop scheduling problem (FJSP) is therefore a representative operational optimization problem in advanced manufacturing. It is also NP-hard, and the search space grows rapidly with the number of jobs, machines, operations, and routing alternatives [5, 11].

Practical workshop schedules are rarely evaluated by a single objective. Minimizing the

makespan improves delivery efficiency, reducing the maximum machine workload alleviates bottleneck resources, and reducing the total workload is related to resource utilization and workload-related consumption. The three objectives are correlated but conflicting: assigning many operations to the fastest machines may shorten completion time but can also increase bottleneck congestion. Multi-objective FJSP is thus a natural decision-support formulation for flexible manufacturing systems, where managers require a set of trade-off schedules instead of a single black-box solution.

Exact optimization methods, such as mixed-integer linear programming and branch-and-bound, can guarantee optimality on small instances, but their computational cost grows exponentially with problem size and limits their use in medium- and large-scale flexible manufacturing environments. Deep reinforcement learning (DRL) has recently been used to learn dispatching policies from graph-structured scheduling states [10, 14–16]. However, DRL methods usually require extensive online interaction with simulated environments, are sensitive to the training instance distribution, and need carefully designed rewards to balance conflicting production objectives. Their step-wise decision form is also less suited to producing a complete Pareto set in a single run. More importantly, the heterogeneous disjunctive graphs used in these dynamic-decision policies mainly encode dynamic quantities, such as processing progress and machine idle time, as node features, leave virtual head and tail nodes out of the embedding, and lack a unified encoding for heterogeneous edge types. Such representations are not directly aligned with the static supervised setting considered in this paper, where the complete FJSP instance is mapped to dispatching-rule priors before the evolutionary search starts. The learning model is therefore used as an offline prior generator. It captures the static mapping from instance structure to promising dispatching strategies and initializes a population-based search without replacing it.

Evolutionary algorithms, especially Non-dominated Sorting Genetic Algorithm II (NSGA-II) [3], remain attractive for multi-objective FJSP because they can produce a set of non-dominated schedules in a single run. Nevertheless, two limitations remain. First, initial populations are often generated randomly or by fixed dispatching rules,

so the mapping from instance structure to promising rule strategies is not explicitly learned. Second, the conventional single-population structure couples convergence and diversity maintenance in one population. Elite preservation may accelerate convergence but can gradually narrow the population distribution. Diversity-preserving operators alone may also fail to exploit high-quality Pareto regions. Existing dual-population designs partly address this issue, but many of them use homogeneous initialization or one-way information transfer, making the auxiliary population a passive follower rather than an active search component. In short, an instance-aware initialization prior and a heterogeneous, bidirectionally coupled dual-population structure have rarely been designed as one method. Learned priors are seldom tailored to populations with different search roles, and existing dual-population search rarely receives prior guidance from instance structure.

To address these limitations, this paper develops an enhanced multi-objective evolutionary optimization approach for FJSP. The blind-initialization limitation is addressed by an offline graph neural network (GNN) that learns an instance-specific distribution over dispatching-rule combinations and turns it into a structure-aware initialization prior. The convergence–diversity coupling is addressed by an online heterogeneous dual-population search, in which a main and an exploratory population consume the same prior in different ways and exchange information bidirectionally to optimize makespan, bottleneck workload, and total workload. The contributions are as follows.

1. **A GNN-based dispatching-rule distribution predictor for multi-objective FJSP (addressing blind initialization).** Each instance is encoded as a static heterogeneous disjunctive graph with virtual head and tail nodes, refined directed edge sets, and unified node and edge features. An edge-conditioned convolution (NNConv) with global attention pooling then regresses a probability distribution over dispatching-rule combinations. Multi-objective weighted soft labels over makespan, bottleneck workload, and total workload supervise the predictor, extending graph-based rule prediction from a single-objective to a multi-objective setting.

## 2. A graph neural network-guided heterogeneous dual-population cooperative evolutionary algorithm with adaptive mixed evolution (GNN-HDPCEA-AME).

Guided by the predicted rule distribution, a heterogeneous main population (MP) initialized for broad Pareto-front coverage and an exploratory population (EP) initialized for intensified search around promising regions evolve in the same optimization process. A bidirectional exchange mechanism and an adaptive mixed evolution (AME) strategy manage convergence and diversity throughout the search.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formulates the multi-objective FJSP. Section 4 presents the proposed data-driven evolutionary framework. Section 5 describes the experimental design. Section 6 reports and discusses the results. Section 7 concludes the paper.

## 2 Related Work

### 2.1 Flexible Job Shop Scheduling in Manufacturing Systems

FJSP extends the classical job shop scheduling problem by allowing each operation to be processed on one of multiple eligible machines. The additional routing flexibility increases modeling realism but also enlarges the search space. Brandimarte [2] introduced representative benchmark instances and solved FJSP by tabu search. Hurink et al. [8] further studied job-shop scheduling with multi-purpose machines. Since then, FJSP has become a standard benchmark problem for manufacturing scheduling and has been extended to integrated scheduling, distributed shops, energy-aware scheduling, and dynamic production environments [11].

In advanced manufacturing contexts, the value of a schedule is not limited to completion time. Workload balance, bottleneck avoidance, and resource utilization are also important for stable shop-floor operation. Multi-objective formulations therefore better represent practical decision-making requirements. Early multi-objective FJSP methods used fuzzy logic, evolutionary search, or hybrid metaheuristics to approximate Pareto trade-offs [9]. Recent work continues to use evolutionary

and swarm intelligence methods because they naturally maintain a population of candidate schedules and can return multiple trade-off solutions in a single run [4, 17].

### 2.2 Learning-Assisted Scheduling and Graph Representations

Learning-based scheduling methods have received increasing attention because manufacturing data and simulation instances can be used to improve dispatching decisions. DRL methods formulate scheduling as a Markov decision process and learn policies through interaction. Zhang et al. [16] learned dispatching rules for job shop scheduling. Song et al. [14] combined graph neural networks with DRL for FJSP, and Lei et al. [10] developed a multi-action DRL framework. More recent heterogeneous graph approaches further model operation-machine relationships and meta-path structures [15]. Most learning-based schedulers operate on a disjunctive graph, which is the standard structural representation of an FJSP instance. In the classical disjunctive graph [12], operation nodes are linked by conjunctive arcs that encode the precedence order within each job, while disjunctive arcs connect operations competing for the same machine, so machine constraints are represented only implicitly. Under machine flexibility, enumerating all such machine-conflict pairs makes the graph dense and hard to process [16]. To alleviate this, Song et al. [14] proposed a heterogeneous disjunctive graph that introduces machine nodes and replaces the disjunctive arcs with operation-machine edges, reducing edge density and making machine-related constraints explicit.

Although these methods demonstrate the value of graph learning, their primary target is online dispatching. The graph state usually changes during schedule construction, and the model selects one operation or machine action at each decision step. Here the learning target is different: the complete FJSP instance is encoded as a static graph and mapped to a distribution over dispatching-rule combinations. The resulting distribution is not a final schedule. It is an instance-aware prior that initializes and guides the evolutionary search. The population-based optimizer still approximates the Pareto front, with learning used only to reduce blind initialization. For graph neural network input, the graph is stored as an edge-index tensor in

which each undirected operation-machine edge is represented by two opposite directed edges, so that messages can be passed in both directions during convolution.

## 2.3 Multi-Objective Evolutionary Algorithms for FJSP

NSGA-II is one of the most widely used multi-objective evolutionary algorithms because of its fast non-dominated sorting and crowding-distance-based selection [3]. It is suitable for FJSP because schedule encodings can be directly evolved and evaluated under multiple objectives. Nevertheless, random initialization and single-population evolution can be inefficient when the search space is large and the Pareto front is irregular.

Dual-population and multi-population evolutionary algorithms attempt to decouple different search roles by maintaining multiple interacting populations. Recent studies have used cooperative populations, local search, simulated annealing, or learning-assisted mechanisms to improve diversity and convergence [7, 17]. However, many dual-population designs still use similar initial distributions across populations or transfer information in a single direction. For multi-objective FJSP, each population should have an explicit search role. One population should preserve broad front coverage, and another should intensify search around promising regions. The proposed heterogeneous dual-population mechanism follows this requirement.

## 3 Problem Description and Multi-Objective Model

### 3.1 FJSP Definition

An FJSP instance is denoted as  $\mathcal{I} = (J, M, O, P)$ , where  $J = \{J_1, \dots, J_n\}$  is the set of jobs,  $M = \{M_1, \dots, M_m\}$  is the set of machines,  $O = \{O_{ij}\}$  is the set of operations, and  $P$  stores the processing times. Job  $J_i$  contains  $n_i$  ordered operations. Operation  $O_{ij}$  can be processed by a subset of eligible machines  $M_{ij} \subseteq M$ . If  $O_{ij}$  is assigned to machine  $M_k$ , its processing time is  $p_{ijk}$ .

The schedule is determined by two coupled decisions: machine assignment and operation sequencing. Let  $x_{ijk} \in \{0, 1\}$  indicate whether operation  $O_{ij}$  is processed on machine  $M_k$ . Each operation

must be assigned to exactly one eligible machine:

$$\sum_{M_k \in M_{ij}} x_{ijk} = 1, \quad \forall O_{ij} \in O. \quad (1)$$

Technological precedence inside each job must be respected:

$$S_{i,j+1} \geq C_{ij}, \quad \forall i, j \quad (2)$$

where  $S_{ij}$  and  $C_{ij}$  are the start and completion times of operation  $O_{ij}$ . Each machine can process at most one operation at a time. The detailed disjunctive non-overlap constraints are handled by the decoding procedure of the evolutionary solver.

### 3.2 Optimization Objectives

The model considers three production objectives. The first objective is the makespan:

$$f_1 = \min C_{\max}, \quad (3)$$

$$C_{\max} = \max_{i,j} C_{ij}. \quad (4)$$

It reflects delivery efficiency and overall completion time.

The second objective is the maximum machine workload:

$$f_2 = \min W_{\max}, \quad (5)$$

$$W_{\max} = \max_k \sum_{i=1}^n \sum_{j=1}^{n_i} p_{ijk} x_{ijk}. \quad (6)$$

It reflects bottleneck resource pressure.

The third objective is the total machine workload:

$$f_3 = \min W_{\text{total}}, \quad (7)$$

$$W_{\text{total}} = \sum_{k=1}^m \sum_{i=1}^n \sum_{j=1}^{n_i} p_{ijk} x_{ijk}. \quad (8)$$

It is used as a workload-related proxy for resource utilization. It should not be interpreted as a complete energy model; instead, it measures the accumulated processing burden selected by machine assignment decisions.

For two feasible schedules  $\mathbf{x}$  and  $\mathbf{y}$ ,  $\mathbf{x}$  Pareto-dominates  $\mathbf{y}$  if all objectives of  $\mathbf{x}$  are no worse than those of  $\mathbf{y}$  and at least one objective is strictly better. The goal is to obtain an approximate Pareto set  $\mathcal{A}$  that is close to the reference front and well distributed across the trade-off surface.

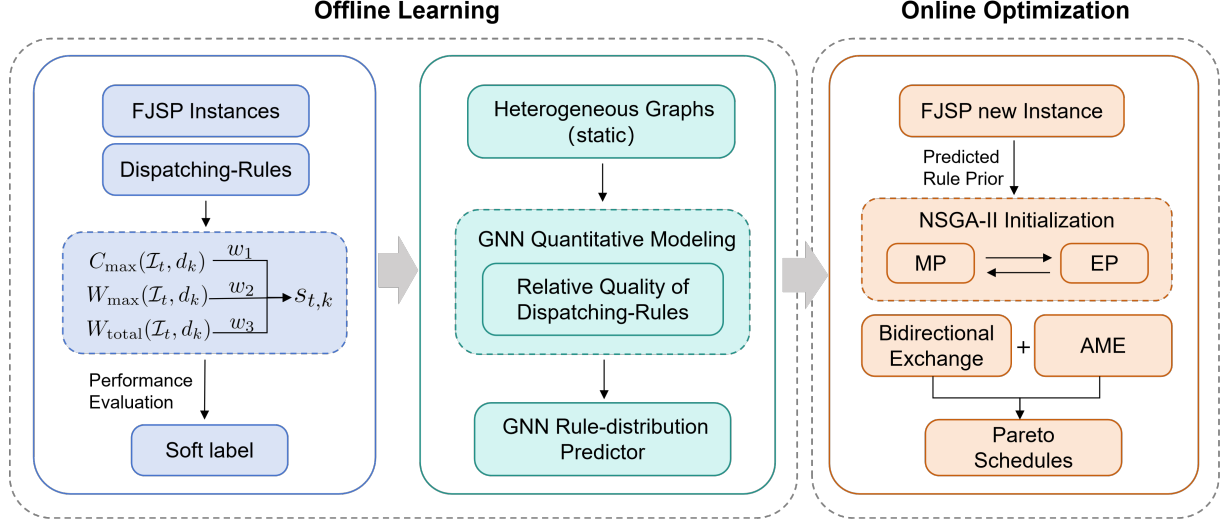


Fig. 1 Overall architecture of the proposed data-driven heterogeneous dual-population evolutionary framework

## 4 Proposed Data-Driven Dual-Population Evolutionary Framework

### 4.1 Overall Architecture

The proposed GNN-HDPCEA-AME framework consists of an offline learning stage and an online optimization stage, as illustrated in Fig. 1. In the offline stage, historical or simulated FJSP instances are converted into static heterogeneous disjunctive graphs. A pool of dispatching-rule combinations is evaluated under the three objectives, and the multi-objective performance of each rule combination is transformed into a soft probability distribution. A GNN is then trained as a graph-level rule-distribution regressor.

In the online optimization stage, a new FJSP instance is first encoded as a graph and passed through the trained GNN. The predicted rule distribution guides the initialization of two heterogeneous populations in a dual-population NSGA-II framework. The MP maintains broad Pareto-front coverage. The EP performs intensified search around promising regions. Bidirectional exchange and adaptive mixed evolution connect the two populations throughout the search. The complete online procedure is summarized in Algorithm 2.

### 4.2 Heterogeneous Disjunctive Graph Representation

An FJSP instance is commonly represented as a disjunctive graph, in which machine competition is encoded by disjunctive arcs and machine constraints remain implicit. To make machine-related information explicit, a heterogeneous disjunctive graph extracts machines as a separate node type and links each operation to its eligible machines through machine-operation arcs weighted by processing time [14]. The heterogeneous representation is mainly designed for the dynamic graph states used in DRL-based dispatching, where node features describe the evolving scheduling status. Because the present work targets a static supervised-learning setting that maps a complete instance to dispatching-rule priors, the existing heterogeneous disjunctive graph is improved from two aspects: structural adjustment and feature encoding.

The improved heterogeneous disjunctive graph is denoted by  $\mathcal{G}^{hetero}(\mathcal{I}) = (\mathcal{V}, \mathcal{E}^{dir} \cup \mathcal{E}^{undir}, X, E)$  and is illustrated in Fig. 2. The node set  $\mathcal{V} = \{v_{start}\} \cup \{v_{end}\} \cup \mathcal{V}_M \cup \mathcal{V}_O$  merges operation nodes  $\mathcal{V}_O$  and machine nodes  $\mathcal{V}_M$  into one set and adds two virtual nodes  $v_{start}, v_{end}$  carrying global start and terminal semantics. The directed edge set  $\mathcal{E}^{dir} = \mathcal{E}_{start} \cup \mathcal{E}_{prec} \cup \mathcal{E}_{end}$  refines the precedence arcs into start edges from  $v_{start}$  to the first operation of each job, precedence edges between



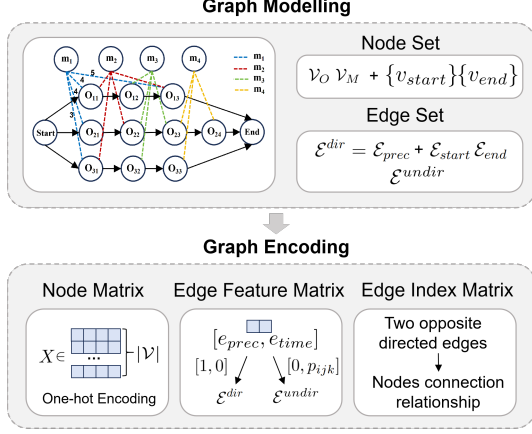


Fig. 2 Heterogeneous disjunctive graph representation

consecutive operations of a job, and terminal edges from the last operation of each job to  $v_{end}$ . The undirected edge set  $\mathcal{E}^{undir}$  connects each operation node to its eligible machine nodes, so machine flexibility is modeled explicitly without enumerating all machine-conflict pairs.

Building on the structural adjustment, a feature encoding is designed for static graph-level learning. The node feature matrix  $X \in \mathbb{R}^{|\mathcal{V}| \times 4}$  adopts a four-dimensional one-hot encoding of the node type, distinguishing start, terminal, machine, and operation nodes, which avoids state-dependent dynamic features and fits the static instance representation. The edge feature matrix  $E$  assigns each edge a two-dimensional vector  $[e_{prec}, e_{time}]$ :  $e_{prec} \in \{0, 1\}$  indicates whether the edge is a directed precedence-type edge, and  $e_{time}$  stores the processing time  $p_{ijk}$  when the edge is a machine-operation edge ( $e_{prec} = 0$ ). For graph neural network input, the graph is stored as an edge-index tensor in which each undirected operation-machine edge is represented by two opposite directed edges, so that messages can be passed in both directions during convolution.

### 4.3 GNN-Based Dispatching-Rule Prior Learning

Let  $\mathbf{h}_v^{(l)}$  be the embedding of node  $v$  at layer  $l$  and  $\mathbf{e}_{uv}$  be the feature of edge  $(u, v)$ . Following the message-passing paradigm [6], edge-conditioned convolution (NNConv) [13] updates node embeddings by generating an edge-specific weight matrix

through a neural network  $\Theta(\cdot)$ :

$$\mathbf{h}_v^{(l+1)} = \sigma \left( \mathbf{W}^{(l)} \mathbf{h}_v^{(l)} + \sum_{u \in \mathcal{N}(v)} \Theta^{(l)}(\mathbf{e}_{uv}) \mathbf{h}_u^{(l)} \right), \quad (9)$$

where  $\mathcal{N}(v)$  denotes the neighbors of  $v$  and  $\sigma(\cdot)$  is a nonlinear activation function. The edge-conditioned update allows relation types and processing-time attributes to modulate message passing.

After two edge-conditioned convolution layers, node embeddings are aggregated by global attention pooling:

$$\mathbf{g} = \sum_{v \in V} a_v \mathbf{h}_v, \quad a_v = \frac{\exp(\phi(\mathbf{h}_v))}{\sum_{u \in V} \exp(\phi(\mathbf{h}_u))}, \quad (10)$$

where  $\phi(\cdot)$  is a learnable gate network. The graph representation  $\mathbf{g}$  is passed to a fully connected regression head followed by softmax normalization to produce  $\hat{\mathbf{p}} \in [0, 1]^K$ , where  $K$  is the number of retained dispatching-rule combinations. The overall prior-learning architecture is shown in Fig. 3.

### 4.4 Multi-Objective Soft-Label Construction

For training instance  $\mathcal{I}_t$  and dispatching-rule combination  $d_k$ , the three objective values are converted into a weighted composite score:

$$s_{t,k} = w_1 C_{\max}(\mathcal{I}_t, d_k) + w_2 W_{\max}(\mathcal{I}_t, d_k) + w_3 W_{\text{total}}(\mathcal{I}_t, d_k), \quad (11)$$

where  $w_1, w_2, w_3 \geq 0$  and  $w_1 + w_2 + w_3 = 1$ . Because  $W_{\text{total}}$  is usually larger in scale than  $C_{\max}$  and  $W_{\max}$ , a smaller  $w_3$  is used to provide implicit scale compensation while preserving its manufacturing meaning.

The composite score is compressed by a logarithmic transform:

$$y_{t,k}^{multi} = \log(s_{t,k} + 1). \quad (12)$$

The target soft label is obtained by a softmax transform:

$$p_{t,k}^{multi} = \frac{\exp(-y_{t,k}^{multi})}{\sum_{l=1}^K \exp(-y_{t,l}^{multi})}. \quad (13)$$

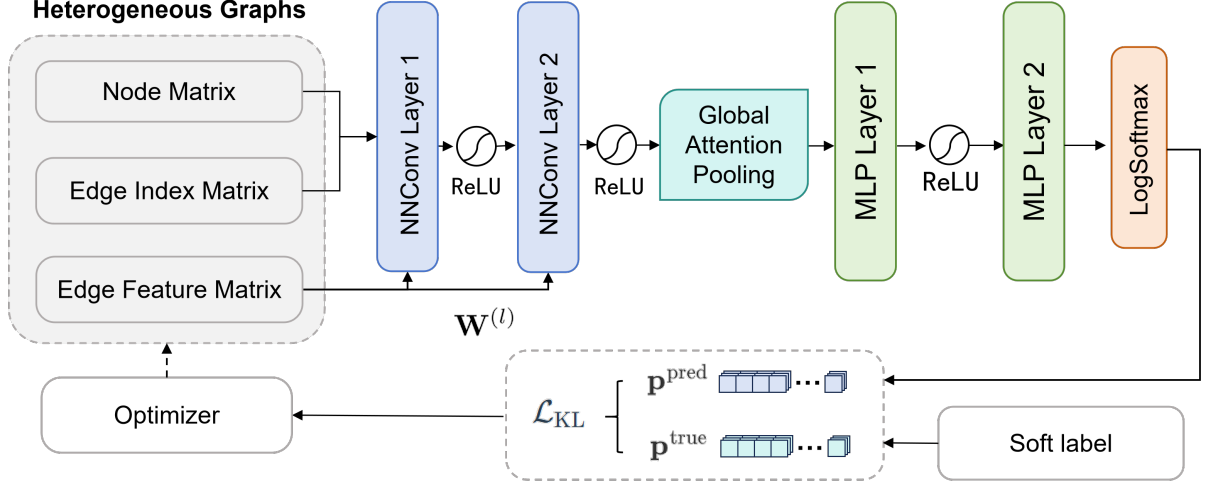


Fig. 3 GNN-based dispatching-rule prior learning

Better rule combinations therefore receive higher probabilities. Competitive non-best rules still retain supervision signals. The GNN is trained by minimizing the Kullback–Leibler divergence between  $\mathbf{p}^{multi}$  and  $\hat{\mathbf{p}}$ .

#### 4.5 Heterogeneous Dual-Population Initialization

The trained GNN outputs a predicted distribution  $\hat{\mathbf{p}}$  that encodes the relative quality of the  $K$  dispatching-rule combinations under the three weighted objectives. Since it remains a  $K$ -dimensional probability vector independent of the number of objectives, the multi-objective preference is carried entirely by  $\hat{\mathbf{p}}$  and can be consumed directly during initialization. To decouple convergence from diversity, two populations with different roles are initialized from the same prior. The Main Population (MP) covers a broad region of the Pareto front. The Exploratory Population (EP) starts near a high-quality core region for intensified search. Because these two roles impose different requirements on the initial population, the shared prior  $\hat{\mathbf{p}}$  is exploited in two distinct ways.

Maintaining broad front coverage calls for an initial population that is diverse and spread over several promising rule combinations instead of concentrated on one. The MP is therefore initialized by a confidence-driven exploitation–exploration strategy (denoted AdaptiveProbEE), which allocates individuals across rule combinations in proportion

to  $\hat{\mathbf{p}}$  and adapts the exploitation ratio to the prediction confidence. The confidence is measured by the normalized entropy of  $\hat{\mathbf{p}}$ :

$$C = 1 - \frac{H(\hat{\mathbf{p}})}{\ln K}, \quad H(\hat{\mathbf{p}}) = - \sum_{k=1}^K \hat{p}_k \ln \hat{p}_k. \quad (14)$$

A high  $C$  indicates a sharp and reliable prediction, so more individuals exploit the high-probability rules. A low  $C$  indicates an uncertain prediction, so the exploitation ratio is reduced and more exploratory combinations are sampled. The resulting MP embeds the instance-aware prior and covers a wide region of the search space. Algorithm 1 summarizes the procedure.

The opposite role, intensifying search around a high-quality core region, calls for a focused seed population located near the best compromise predicted for the instance. The EP is therefore initialized from the single rule combination with the highest predicted probability ( $k^* = \arg \max_k \hat{p}_k$ , denoted BestSingleRule), forming a near-homogeneous population in which every individual except the first is slightly perturbed in its operation-sequence and machine-selection encodings to avoid exact duplication. Since the top-predicted rule usually corresponds to the best overall multi-objective compromise for the instance, this focused initialization provides the EP with a high-quality core from which to perform deep neighborhood exploitation.

---

**Algorithm 1** AdaptiveProbEE: confidence-driven initialization of MP

---

**Require:** predicted distribution  $\hat{\mathbf{p}}$ , population size  $N_{MP}$ , confidence threshold  $\tau$ , base ratios and spans  $(r_{\min}^h, \Delta r^h, r_{\min}^l, \Delta r^l)$ , perturbation rates  $(\rho_{\text{exploit}}^h, \rho_{\text{exploit}}^l, \rho_{\text{explore}})$

**Ensure:** Main population MP

- 1: Compute confidence  $C \leftarrow 1 - H(\hat{\mathbf{p}})/\ln K$
  - 2: **if**  $C > \tau$  **then**  $\triangleright$  high confidence: trust the prior
    - 3:  $r_{\text{exploit}} \leftarrow r_{\min}^h + \Delta r^h (C - \tau)/(1 - \tau)$
    - 4:  $\rho_{\text{exploit}} \leftarrow \rho_{\text{exploit}}^h$
  - 5: **else**  $\triangleright$  low confidence: favor exploration
    - 6:  $r_{\text{exploit}} \leftarrow r_{\min}^l + \Delta r^l C/\tau$
    - 7:  $\rho_{\text{exploit}} \leftarrow \rho_{\text{exploit}}^l$
  - 8: **end if**
  - 9:  $N_{\text{exploit}} \leftarrow \lfloor r_{\text{exploit}} N_{MP} \rfloor$
  - 10:  $N_{\text{explore}} \leftarrow N_{MP} - N_{\text{exploit}}$
  - 11: **Exploit:** allocate  $N_{\text{exploit}}$  individuals to rules in proportion to  $\hat{\mathbf{p}}$ , decode each, and apply mild perturbation  $\rho_{\text{exploit}}$
  - 12: **Explore:** draw  $N_{\text{explore}}$  rules uniformly at random, decode each, and apply strong perturbation  $\rho_{\text{explore}}$
  - 13:  $\text{MP} \leftarrow \text{Shuffle}(\text{exploit} \cup \text{explore})$
  - 14: **return** MP
- 

#### 4.6 Bidirectional Population Exchange and Adaptive Mixed Evolution

If the two populations evolved in isolation, useful search information would remain separated. The MP would not receive the near-front solutions discovered by the EP's local exploitation. The EP would also remain close to its homogeneous seed region and would have limited access to sparse parts of the front. A bidirectional exchange is therefore introduced for periodic information sharing.

Every exchange period, the top 10% elite individuals from EP are injected into MP. The injected individuals are selected from the best non-dominated fronts of EP and replace individuals in the worst front of MP. The EP-to-MP transfer helps MP absorb high-quality solutions discovered by local exploitation. In the opposite direction, 5% of MP individuals with large crowding distances are injected into EP as diversity anchors. The selected individuals usually lie in sparse regions of

---

**Algorithm 2** GNN-HDPCEA-AME for multi-objective FJSP

---

**Require:** FJSP instance  $\mathcal{I}$ , trained GNN predictor, population sizes  $N_{MP}, N_{EP}$ , maximum generation  $G$

**Ensure:** Approximate Pareto schedule set  $\mathcal{A}$

- 1: Encode  $\mathcal{I}$  as static heterogeneous graph  $\mathcal{G}^h$
  - 2: Predict rule distribution  $\hat{\mathbf{p}} \leftarrow \text{GNN}(\mathcal{G}^h)$
  - 3: Compute prediction confidence  $C$  from  $\hat{\mathbf{p}}$
  - 4: Initialize MP by AdaptiveProbEE using  $\hat{\mathbf{p}}$
  - 5: Initialize EP by BestSingleRule using  $\hat{\mathbf{p}}$
  - 6: **for**  $g = 1$  to  $G$  **do**
    - 7: Generate MP offspring using NSGA-II variation and decoding
    - 8: Update MP by non-dominated sorting and crowding distance
    - 9: Generate EP offspring by adaptive mixed evolution
    - 10: Update EP by non-dominated sorting and crowding distance
    - 11: **if**  $g$  is an exchange generation **then**
      - 12: Inject EP elites into MP
      - 13: Inject MP diversity anchors into EP
    - 14: **end if**
  - 15: **end for**
  - 16:  $\mathcal{A} \leftarrow$  non-dominated solutions in  $\text{MP} \cup \text{EP}$
  - 17: **return**  $\mathcal{A}$
- 

the current front and provide new neighborhoods for EP to explore. The exchange ratio is asymmetric because the MP is the primary source of the final Pareto front and benefits from EP elites. The EP relies mainly on its own evolution and needs only a few diverse anchors to redirect its search.

Because the EP is initialized from a single rule combination, its individuals are highly similar, and standard crossover would be ineffective, since recombining near-identical parents rarely yields meaningful new solutions. The EP therefore requires an offspring-generation mechanism that does not rely on parent diversity. Adaptive mixed evolution (AME) meets this need by generating EP offspring from three sources: random injection, neighborhoods of EP elites, and neighborhoods of MP front individuals. The source ratios  $(\alpha_{\text{rand}}, \alpha_{\text{ep}}, \alpha_{\text{mp}})$  are set piecewise according to the evolutionary progress  $g/G$ , as summarized in Table 1, with  $\alpha_{\text{mp}} = 1 - \alpha_{\text{rand}} - \alpha_{\text{ep}}$ . The numbers



**Table 1** Adaptive source ratios of AME by evolutionary stage

| Stage $g/G$ | $\alpha_{\text{rand}}$ | $\alpha_{\text{ep}}$ | $\alpha_{\text{mp}}$ |
|-------------|------------------------|----------------------|----------------------|
| [0, 0.4)    | 0.15                   | 0.30                 | 0.55                 |
| [0.4, 0.8)  | 0.08                   | 0.25                 | 0.67                 |
| [0.8, 1]    | 0.05                   | 0.20                 | 0.75                 |

of offspring drawn from the three sources are

$$\begin{aligned} n_{\text{rand}} &= \max(1, \lfloor N_{EP} \alpha_{\text{rand}} \rfloor), \\ n_{\text{ep}} &= \max(1, \lfloor N_{EP} \alpha_{\text{ep}} \rfloor), \\ n_{\text{mp}} &= N_{EP} - n_{\text{rand}} - n_{\text{ep}}. \end{aligned} \quad (15)$$

As the search proceeds, the shares of random injection and EP-elite neighborhoods decrease, and the share of MP-front neighborhoods increases. Early generations rely more on random injection and EP-elite perturbation to break the initial homogeneity of EP. Later generations shift more offspring toward the MP front to refine the search around its stabilized, diverse front.

The six neighborhood operators cover both encoding dimensions: three act on the operation sequence with increasing perturbation strength (adjacent swap, insertion, and segment reversal), two act on machine selection (a random single-operation reassignment and a bottleneck-machine batch reassignment that relieves the most-loaded machine), and one perturbs both dimensions in a coupled move. One operator is selected at random for each offspring, enabling the EP to search sequencing and routing decisions simultaneously.

The overall procedure is illustrated in Fig. 4: the MP and EP evolve in parallel through NSGA-II variation and AME, respectively, periodically reinforce each other through the bidirectional exchange, and are finally merged and non-dominated-sorted to produce the output Pareto schedules.

## 5 Experimental Design

### 5.1 Datasets and Implementation

The GNN predictor was trained and validated on a synthetic FJSP instance set generated by randomized manufacturing parameters. The number of jobs was sampled from  $\{5, 10, 15, 20\}$ , the number of machines from  $\{4, 6, 8, 10\}$ , the flexibility ratio was set to 0.5, and processing times were sampled

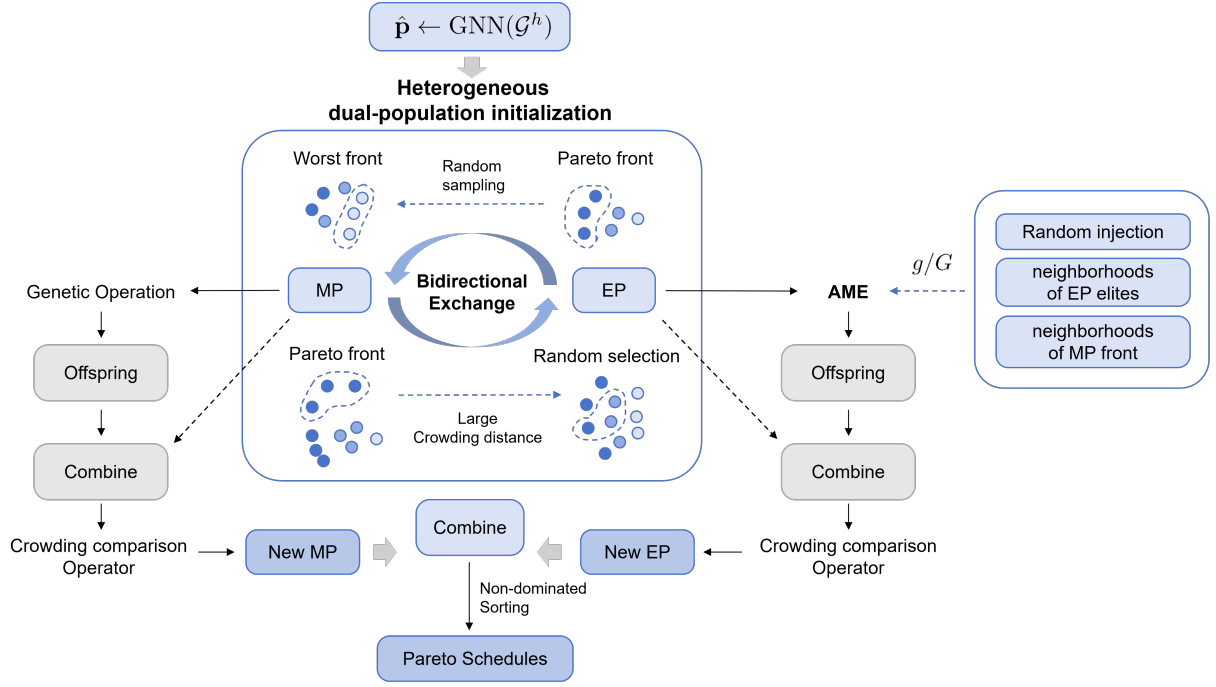
from  $[1, 100]$ . A total of 1000 instances were generated and split into training and validation sets by stratified sampling. The split is performed at the instance level, so the same scheduling instance never appears in both the training and validation subsets. The public benchmark is used only for external evaluation after model selection, without retraining the GNN predictor or re-tuning the evolutionary parameters.

Public benchmark evaluation used BehnkeGeiger instances [1]. Hurink-rdata instances [8] were not used in the multi-objective evaluation because their processing-time structure leads to limited discrimination in the  $W_{\text{total}}$  objective under the current objective definition.

The dispatching-rule pool was constructed from job-sequencing rules and machine-selection rules adopted from Lei et al. [10]. After filtering low-frequency rule combinations, five combinations were retained as GNN output classes: FIFO+SPT, MOPNR+SPT, MOPNR+EET, MWKR+SPT, and MWKR+EET. For each instance, every candidate rule combination seeds a standard NSGA-II, and the averaged three-objective performance over  $R$  runs is mapped to a soft label via the construction of Section 4.4, giving the (instance graph, soft-label) training pairs. The parameter, initialization, and GNN-training settings used in the experiments are summarized in Table 2. The AdaptiveProbEE values are set according to the role of each term instead of per-instance tuning, and are kept fixed across all benchmarks without re-tuning on the public test sets. All experiments were conducted in a Linux-based container environment. The server was equipped with two Intel Xeon Gold 6248R CPUs, 251 GB memory, and two NVIDIA RTX 3090 GPUs. The implementation used Python 3.8.6 and PyTorch 1.7.1 with CUDA 11.0. The multi-objective comparison was carried out on a set of 20 validation instances and on the public BehnkeGeiger instances. Each stochastic algorithm was independently repeated 10 times, and the reported values are means taken across instances and runs.

### 5.2 Evaluation Metrics

The quality of the multi-objective solutions is measured by four metrics. Hypervolume (HV) reflects both convergence and coverage and is



**Fig. 4** Workflow of Bidirectional population exchange and Adaptive Mixed Evolution (AME) of the EP

larger when the front dominates a larger objective-space volume. Its reference point is set to  $\mathbf{r} = 1.1 \cdot (\max_i f_1^{(i)}, \max_i f_2^{(i)}, \max_i f_3^{(i)})$ , i.e.  $1.1 \times$  the per-objective maxima taken over the merged non-dominated solutions of all methods compared in the same experiment. The inverted generational distance (IGD) is computed against a reference front merged from the non-dominated solutions of all compared methods, by averaging the distance from each point of this reference front to the obtained front. Smaller values indicate a closer approximation. Because both the HV reference point  $\mathbf{r}$  and the IGD reference front are built from the union of the methods compared within a given experiment, a table that includes methods with more extended fronts pushes  $\mathbf{r}$  outward and thereby inflates the HV magnitudes of *all* entries in that table. Consequently the absolute HV and IGD values are comparable within each table but not directly across tables that correspond to different method sets. Section 6.3 returns to this point when interpreting the ablation tables. Spacing  $\Delta$  characterizes the uniformity of the solution distribution along the front, and the spacing variance  $\Delta_v$  measures the stability of this distribution across

repeated runs. Smaller values of both are preferable. Together they cover convergence, proximity, distribution uniformity, and stability. Thus, the four metrics are not interchangeable: HV and IGD evaluate the global quality of the obtained front, whereas  $\Delta$  and  $\Delta_v$  check whether that front is evenly and repeatably distributed. This distinction is important for scheduling use, because a front with similar proximity can still be less useful if most schedules concentrate around only one production preference.

The prediction quality of the GNN is examined from three aspects, each with one representative metric. Distribution-fitting quality is assessed by the Jensen–Shannon (JS) divergence, cosine similarity, and Brier score, which together quantify how closely the predicted distribution matches the target soft label in shape and magnitude. Advantage-rule-set matching is assessed by the Top-3 match rate (Top-3 Match), which measures whether the most competitive rule combinations are captured among the top predictions. Best-rule hit is assessed by the Top-3 accuracy (Top-3 Accuracy), which measures whether the truly best rule combination falls within the top predicted candidates. The three aspects respectively reflect overall

**Table 2** Parameter and hyperparameter settings used in the experiments

| Parameter                                             | Symbol             | Value |
|-------------------------------------------------------|--------------------|-------|
| <i>Performance-label evaluation (offline NSGA-II)</i> |                    |       |
| Population size                                       | $N_{pop}$          | 100   |
| Maximum generations                                   | $G$                | 50    |
| Independent runs                                      | $R$                | 10    |
| Crossover probability                                 | $p_c$              | 0.8   |
| Mutation probability                                  | $p_m$              | 0.1   |
| <i>AdaptiveProbEE initialization</i>                  |                    |       |
| Confidence threshold                                  | $\tau$             | 0.3   |
| High-conf. base ratio                                 | $r_{min}^h$        | 0.70  |
| High-conf. ratio span                                 | $\Delta r^h$       | 0.20  |
| Low-conf. base ratio                                  | $r_{min}^l$        | 0.40  |
| Low-conf. ratio span                                  | $\Delta r^l$       | 0.30  |
| High-conf. exploit perturb.                           | $\rho_{exploit}^h$ | 0.10  |
| Low-conf. exploit perturb.                            | $\rho_{exploit}^l$ | 0.25  |
| Explore perturbation                                  | $\rho_{explore}$   | 0.30  |
| <i>GNN training</i>                                   |                    |       |
| Hidden dimension                                      | $d_h$              | 64    |
| Training epochs                                       | $epoch$            | 100   |
| Initial learning rate                                 | $\eta$             | 0.001 |
| Gradient-accum. steps                                 | $N_{batch}$        | 32    |
| Dropout rate                                          | $p_{drop}$         | 0.5   |
| LR-decay patience                                     | $patience$         | 8     |
| LR decay factor                                       | $\gamma_{lr}$      | 0.5   |
| Train split ratio                                     | $r_{train}$        | 0.8   |

distribution fidelity, the coverage of competitive rules, and the reliability of identifying the single best rule.

### 5.3 Research Questions

The experiments are organized around the proposed framework and its two core components, proceeding from the overall framework to the individual components, and are guided by the following research questions.

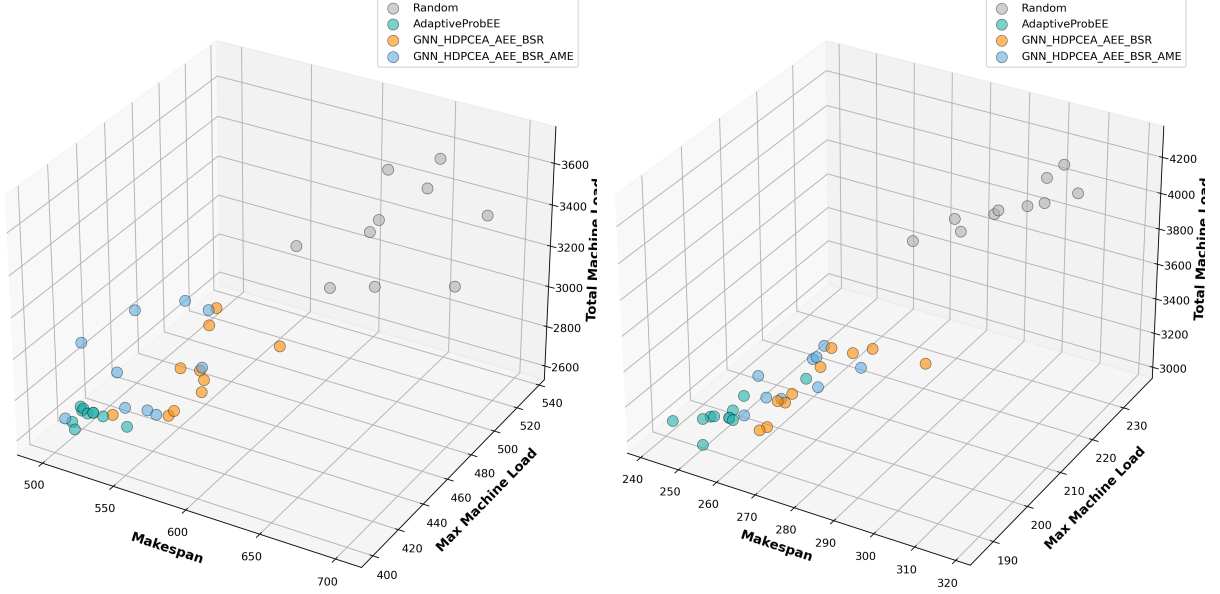
1. *RQ1*. Does the complete GNN-HDPCEA-AME framework achieve overall advantages on multi-objective FJSP (Section 6.1)?
2. *RQ2*. Does the graph-based prior-learning model provide higher prediction quality (Section 6.2)?
3. *RQ3*. Is the heterogeneous dual-population structure reasonable and advantageous (Section 6.3)?

## 6 Results and Discussion

### 6.1 Overall Framework Performance

On both the custom validation set and the BehnkeGeiger benchmark, the full GNN-HDPCEA-AME is compared with single-population NSGA-II (random and AdaptiveProbEE initialization) and the dual-population baseline without AME (GNN-HDPCEA-AEE-BSR, in which AEE denotes the AdaptiveProbEE-initialized MP and BSR the BestSingleRule-initialized EP). The maximum generation is fixed to  $G = 100$  for all methods. The dual-population methods use  $N_{MP} = N_{EP} = 200$ , and the single-population baselines use a population of 200 individuals, so every population is allocated the same per-population budget. Thus, the dual-population methods use about twice the total number of objective evaluations of the single-population baselines. The comparison evaluates the complete framework under its designed cooperative two-population budget. The different total evaluation budget is reported explicitly instead of hidden in the algorithmic description. As shown in Table 3, GNN-HDPCEA-AME attains the best HV, IGD,  $\Delta$ , and  $\Delta_v$  among the compared methods on both datasets, whose three-objective Pareto fronts are visualized in Fig. 5. On the custom validation set, its HV improves over GNN-HDPCEA-AEE-BSR, AdaptiveProbEE, and random NSGA-II by 15.3%, 30.7%, and 192.5%, respectively. Relative to GNN-HDPCEA-AEE-BSR,  $\Delta$  and  $\Delta_v$  are reduced by 36.8% and 31.5%. Its IGD (16.82) is essentially tied with AdaptiveProbEE (16.85). The latter, as a single-objective-oriented initialization, already concentrates near several single-objective optima, so its proximity to the merged reference front is comparable. Its much larger  $\Delta$  and  $\Delta_v$  still indicate a less uniform and less stable front. On the custom set, the learned initialization, the heterogeneous dual-population structure, and AME improve front quality, distribution, and stability. GNN-HDPCEA-AME is uniformly best on both datasets. The single-population baselines behave differently across the two datasets, which helps separate the roles of the learned prior and the dual-population structure.

A closer look at the public benchmark clarifies an apparent anomaly. On BehnkeGeiger, AdaptiveProbEE-seeded single-population NSGA-II reaches a lower HV than random NSGA-II (1.62



**Fig. 5** Three-objective Pareto fronts of GNN-HDPCEA-AME with the custom validation set on the left and the BehnkeGeiger benchmark on the right

vs  $2.02 \times 10^6$ ) but a far smaller IGD (13.70 vs 846.66). The two metrics capture different aspects: HV rewards the extent of the dominated region, so a widely scattered yet poorly converged random front can dominate a larger axis-aligned volume. The prior concentrates the single population around high-quality compromise regions and thus improves proximity (IGD) at the cost of raw spread. Because the BehnkeGeiger instances also deviate from the synthetic training distribution, the standalone prior is less able to broaden coverage on this benchmark. Embedding the prior in the heterogeneous dual-population framework removes this limitation. The breadth-oriented MP and the depth-oriented EP, linked by bidirectional

exchange and AME, restore coverage and then surpass the baselines. GNN-HDPCEA-AME therefore attains both the best HV ( $3.61 \times 10^6$ ) and the best IGD (0.7372) among the compared methods on this benchmark. The learned prior therefore serves as an informative initialization whose full benefit is realized through dual-population cooperation, not as a standalone HV booster.

## 6.2 Effectiveness of Graph-Based Prior Learning

The GNN predictor is trained on the multi-objective performance-label dataset. The samples are stratified by their best-performing rule class

**Table 3** Multi-objective performance on the custom validation set and the BehnkeGeiger benchmark

| Dataset      | Method                 | HV $\uparrow$                        | IGD $\downarrow$ | $\Delta$ $\downarrow$ | $\Delta_v$ $\downarrow$ |
|--------------|------------------------|--------------------------------------|------------------|-----------------------|-------------------------|
| Custom       | Random NSGA-II         | $1.73 \times 10^7$                   | 726.89           | 14.85                 | 7.76                    |
|              | AdaptiveProbEE NSGA-II | $3.87 \times 10^7$                   | 16.85            | 12.01                 | 4.25                    |
|              | GNN-HDPCEA-AEE-BSR     | $4.39 \times 10^7$                   | 29.63            | 5.54                  | 2.03                    |
|              | GNN-HDPCEA-AME         | <b><math>5.06 \times 10^7</math></b> | <b>16.82</b>     | <b>3.50</b>           | <b>1.39</b>             |
| BehnkeGeiger | Random NSGA-II         | $2.02 \times 10^6$                   | 846.66           | 5.9541                | 3.2623                  |
|              | AdaptiveProbEE NSGA-II | $1.62 \times 10^6$                   | 13.70            | 5.9292                | 3.4047                  |
|              | GNN-HDPCEA-AEE-BSR     | $3.51 \times 10^6$                   | 14.06            | 2.2768                | 1.0321                  |
|              | GNN-HDPCEA-AME         | <b><math>3.61 \times 10^6</math></b> | <b>0.7372</b>    | <b>1.3656</b>         | <b>0.5811</b>           |

Note: Arrows indicate the preferred direction ( $\downarrow$  smaller /  $\uparrow$  larger is better), and bold marks the best value in each column (ties bolded jointly). The same conventions apply to the subsequent result tables.

**Table 4** Multi-objective prediction quality on the validation set

| Model     | JS ↓          |               | Cosine ↑      |               | Brier ↓       |               | Top3-Match ↑  |               | Top3-Acc ↑    |               |
|-----------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
|           | Best          | Mean          | Best          | Mean          | Best          | Mean          | Best          | Mean          | Best          | Mean          |
| Att.      | <b>0.0723</b> | <b>0.0751</b> | <b>0.8981</b> | <b>0.8959</b> | <b>0.0136</b> | <b>0.0139</b> | 0.8150        | 0.8067        | <b>0.8736</b> | <b>0.8633</b> |
| SAG       | 0.0743        | 0.0779        | 0.8959        | 0.8924        | 0.0139        | 0.0143        | <b>0.8260</b> | <b>0.8132</b> | <b>0.8736</b> | 0.8630        |
| Set2Set   | 0.0732        | 0.0766        | 0.8934        | 0.8906        | 0.0143        | 0.0146        | 0.8186        | 0.8058        | 0.8681        | 0.8626        |
| Mean      | 0.0779        | 0.0921        | 0.8915        | 0.8910        | 0.0143        | 0.0147        | 0.8040        | 0.7993        | 0.8681        | 0.8614        |
| Deep-Att. | 0.1116        | 0.1541        | 0.8772        | 0.8331        | 0.0165        | 0.0230        | 0.7948        | 0.7158        | 0.8735        | 0.8612        |

Note: Att./Deep-Att./Set2Set/SAG/Mean denote NNConv with attention, deep (3-layer) attention, Set2Set, SAG, and mean pooling, respectively. For each metric, “Best” is the best value over all epochs and “Mean” the average over epochs 51–100. Top3-Match and Top3-Acc are near their ceiling under five rule classes, so the epoch-averaged Mean is more discriminative than Best.

and split into training and validation sets at a ratio of  $r_{train} = 0.8$ , keeping the per-class proportions consistent across both subsets. Each step processes a single graph instance, and an equivalent batch is emulated by gradient accumulation. After every epoch, the model is evaluated over the whole validation set, the learning-rate scheduler decides whether to decay the rate, and the checkpoint with the lowest validation loss is retained. The random seed is fixed to 42. The main GNN hyperparameters are also listed in Table 2. All compared graph-readout and convolution-depth variants share the same input, soft labels, and training protocol, so that the differences in Table 4 reflect the architecture alone.

Table 4 reports both the best value over training (Best) and the converged-stage mean over epochs 51–100 (Mean). NNConv-Attention attains the best result on the three distribution-fitting metrics (JS, Cosine, and Brier) under both Best and Mean, and also the best Top-3 accuracy (best Mean and tied-best Best). On Top-3 Match, NNConv-SAG is marginally higher under both statistics (by about 0.01). Its hard node selection happens to favor a few top-ranked rules, but this slight edge does not carry over to the other metrics. Because Top-3 Match and Top-3 accuracy are close to their ceiling under five rule classes, the models differ little on these two metrics—especially on Best—so the converged Mean better reflects their stable behavior. NNConv-Deep-Attention is consistently the worst on both statistics (its JS Mean rises to 0.1541 and its Cosine Mean drops to 0.8331), confirming that simply increasing depth tends to overfit at the current data scale and supporting the two-layer design. Because the downstream evolutionary search benefits from a calibrated distribution over competitive rules instead of only the single best rule, the attention readout that preserves all node contributions is preferable.

From a manufacturing decision-support perspective, the GNN does not output a final schedule. It reduces blind initialization by identifying rule combinations that suit the structure of a new FJSP instance, letting the evolutionary solver start from a more informed region and retain the ability to explore trade-off schedules.

### 6.3 Rationality of Heterogeneous Population Roles

The role-analysis experiment uses the same four Pareto-front metrics as Section 6.1 (HV, IGD,  $\Delta$ ,  $\Delta_v$ ). In addition, for the single-population initialization strategies the per-objective averages of makespan, maximum machine workload, and total workload (Table 5) are reported to identify which initialization suits the breadth-oriented MP and which the core-focused EP. The dual-population combinations are then compared on the four front metrics (Table 6). The RQ1 comparison and the RQ3 role-analysis experiments are independent experimental batches with different compared strategy sets and independently generated stochastic runs. Repeated strategy names such as Random or AdaptiveProbEE are therefore within-table controls. Their HV, IGD,  $\Delta$ , and  $\Delta_v$  values should be interpreted within the table in which they appear, not compared across Tables 3, 5, and 6.

Table 5 first reports single-population NSGA-II under different initialization strategies. AdaptiveWeighted samples rule combinations according to the learned prior weights. Uniform-Mixed mixes prior-weighted and uniformly sampled rule choices. BestSingleRule uses the highest-probability rule combination, and AdaptiveProbEE is the confidence-driven exploitation–exploration strategy defined in Section 4.5. BestSingleRule is not the strongest on any single objective, but it attains the best HV and the best



**Table 5** Single-population NSGA-II under different initialization strategies on the validation set

| Initialization   | HV $\uparrow$                        | $\Delta$ $\downarrow$ | $\Delta_v$ $\downarrow$ | IGD $\downarrow$ | Mksp* $\downarrow$ | MaxLoad* $\downarrow$ | TotalLoad* $\downarrow$ |
|------------------|--------------------------------------|-----------------------|-------------------------|------------------|--------------------|-----------------------|-------------------------|
| Random           | $1.93 \times 10^7$                   | <b>12.7339</b>        | 5.8543                  | 703.71           | 770.35             | 624.10                | 3484.15                 |
| AdaptiveWeighted | $4.28 \times 10^7$                   | 13.4858               | 5.5609                  | 34.21            | 697.60             | 576.05                | 2923.25                 |
| UniformMixed     | $4.00 \times 10^7$                   | 13.3504               | 5.4539                  | 31.22            | 699.65             | 578.05                | 2923.10                 |
| BestSingleRule   | <b><math>4.62 \times 10^7</math></b> | 13.1510               | <b>4.7173</b>           | 44.43            | 713.10             | 576.00                | 2935.70                 |
| AdaptiveProbEE   | $3.61 \times 10^7$                   | 12.9599               | 5.0782                  | <b>24.12</b>     | <b>687.60</b>      | <b>563.75</b>         | <b>2920.80</b>          |

\* Mksp, MaxLoad, and TotalLoad denote the average of the best (minimum) makespan, maximum machine load, and total load over independent runs, respectively (lower is better).

$\Delta_v$ , indicating a well-covered and stable compromise front. AdaptiveProbEE attains the best IGD and explores the single-objective directions more broadly. The contrast motivates the heterogeneous pairing: AdaptiveProbEE for the breadth-oriented MP and BestSingleRule for the core-focused EP.

Table 6 then compares dual-population variants under an equal per-population budget ( $N_{MP} = N_{EP} = 200$ ,  $G = 100$ ). SFM denotes a temperature-smoothed prior-weighted initialization (SoftmaxTemperature), a smoothed variant of the prior-weighted AdaptiveWeighted idea. The suffix order denotes the MP/EP assignment. As in Section 6.1, each dual-population variant has two populations and therefore uses about twice the total evaluation budget of the single-population positioning rows. Across the four combinations, HV varies from  $1.15 \times 10^8$  to  $1.39 \times 10^8$  and IGD spans roughly fivefold (44.6 to 224.2), showing that the performance gap stems from the MP/EP functional pairing rather than the per-population size. The single-population rows (AdaptiveProbEE, Random) carry different metric values from the same method names in Table 5 because they are independent reruns within this dual-population comparison batch. The HV values are also affected by the broader method set in Table 6. Because this table contains the more extended dual-population fronts, the shared reference point  $\mathbf{r} = 1.1 \cdot (\max f_1, \max f_2, \max f_3)$  is pushed outward and inflates every HV in this table. The two single-population rows therefore serve only as positioning references within Table 6 and are not directly comparable to Table 5, including for  $\Delta$  and  $\Delta_v$ . The adopted GNN-HDPCEA-AEE-BSR combines a large HV ( $1.36 \times 10^8$ ) with a low IGD (46.99). GNN-HDPCEA-SFM-AEE attains a slightly higher HV but an IGD above 224, indicating a broader yet less accurate front. Bidirectional exchange lets the two populations share search information, and the role separation gives the final schedule set coverage

across priorities such as urgent delivery, bottleneck relief, and workload balance.

## 6.4 Manufacturing Implications and Limitations

The proposed framework can be used as an offline-trained and online-applied scheduling decision-support module. Historical or simulated scheduling instances train the GNN predictor offline. For a new production order set, the trained predictor provides instance-aware dispatching-rule priors, and the dual-population solver returns trade-off schedules for delivery urgency, bottleneck machine availability, or workload-balance requirements.

The current study also has limitations. First, the experiments are based on synthetic and public benchmark instances rather than a live production line. Second,  $W_{\text{total}}$  is a workload-related proxy and does not capture all components of energy consumption, such as machine idle energy, tool change, or time-of-use electricity tariffs. Third, the current framework focuses on static FJSP. Dynamic arrivals, machine breakdowns, and urgent order insertion are not yet included. Fourth, the evaluation isolates each component against single-population and reduced dual-population variants. Broader benchmarking against NSGA-III, MOEA/D, and a graph-free predictor, together with formal significance testing, is left to future work.

## 7 Conclusions

The paper developed a heterogeneous dual-population evolutionary optimization approach for multi-objective FJSP. The method uses an instance-specific learned prior to improve initialization and guide the later search. A static heterogeneous disjunctive graph and an edge-conditioned GNN predictor learn dispatching-rule distributions from offline scheduling data. The

**Table 6** Single- vs. dual-population NSGA-II with different MP/EP combinations on the validation set

| Strategy                | HV $\uparrow$                        | $\Delta$ $\downarrow$ | $\Delta_v$ $\downarrow$ | IGD $\downarrow$ |
|-------------------------|--------------------------------------|-----------------------|-------------------------|------------------|
| AdaptiveProbEE (single) | $9.79 \times 10^7$                   | 24.3051               | 9.5747                  | 85.31            |
| Random (single)         | $4.25 \times 10^7$                   | <b>21.2775</b>        | <b>7.1460</b>           | 1565.19          |
| GNN-HDPCEA-BSR-AEE      | $1.15 \times 10^8$                   | 27.4785               | 18.4111                 | <b>44.56</b>     |
| GNN-HDPCEA-SFM-AEE      | <b><math>1.39 \times 10^8</math></b> | 22.4481               | 11.2872                 | 224.21           |
| GNN-HDPCEA-AEE-BSR      | $1.36 \times 10^8$                   | 26.1621               | 15.5279                 | 46.99            |
| GNN-HDPCEA-AEE-SFM      | $1.16 \times 10^8$                   | 27.5454               | 12.7385                 | 223.45           |

learned distribution initializes a main population for Pareto-front breadth and an exploratory population for intensified search around a high-quality core. Bidirectional exchange and AME connect the two populations during optimization.

Experiments on synthetic instances and the public BehnkeGeiger benchmark showed that the approach improved Pareto-front quality, distribution uniformity, and cross-run stability compared with single-population and dual-population baselines. On the custom validation set, the full framework improved HV by 15.3% over the dual-population baseline without AME, by 30.7% over the single-population learned prior, and by 192.5% over random NSGA-II. It also reduced  $\Delta$  and  $\Delta_v$  by 36.8% and 31.5%. Instance-specific prior learning and heterogeneous dual-population search therefore provide useful gains for multi-objective evolutionary scheduling.

Future work will extend the framework to dynamic FJSP with machine breakdowns, order arrivals, and changing processing conditions. More realistic energy-consumption objectives can also be incorporated beyond workload-based proxies, and learned scheduling priors can be combined with explainable decision-support interfaces.

## References

- [1] Behnke D, Geiger MJ (2012) Test instances for the flexible job shop scheduling problem with work centers. Helmut Schmidt University, Institute of Production Management and Logistics
- [2] Brandimarte P (1993) Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research* 41(3):157–183. <https://doi.org/10.1007/BF02023073>
- [3] Deb K, Pratap A, Agarwal S, et al (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6(2):182–197. <https://doi.org/10.1109/4235.996017>
- [4] Gao K, Cao Z, Zhang L, et al (2019) A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems. *IEEE/CAA Journal of Automatica Sinica* 6(4):904–916. <https://doi.org/10.1109/JAS.2019.1911540>
- [5] Garey MR, Johnson DS, Sethi R (1976) The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research* 1(2):117–129. <https://doi.org/10.1287/moor.1.2.117>
- [6] Gilmer J, Schoenholz SS, Riley PF, et al (2017) Neural message passing for quantum chemistry. In: *International Conference on Machine Learning*, pp 1263–1272
- [7] Guo Q, Qiu Y, Shi Y, et al (2025) An enhanced dual-population NSGA-II for solving multi-objective emergency supplies dispatch problem based on truck-drone-dispatch vehicle collaboration. *Computers & Operations Research* 177:107324. <https://doi.org/10.1016/j.cor.2025.107324>
- [8] Hurink J, Jurisch B, Thole M (1994) Tabu search for the job-shop scheduling problem with multi-purpose machines. *Operations-Research-Spektrum* 15(4):205–215. <https://doi.org/10.1007/BF01719451>
- [9] Kacem I, Hammadi S, Borne P (2002) Pareto-optimality approach for flexible job-shop

scheduling problems: hybridization of evolutionary algorithms and fuzzy logic. *Mathematics and Computers in Simulation* 60(3–5):245–276. [https://doi.org/10.1016/S0378-4754\(02\)00019-8](https://doi.org/10.1016/S0378-4754(02)00019-8)

- [10] Lei K, Guo P, Zhao W, et al (2022) A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem. *Expert Systems with Applications* 205:117796. <https://doi.org/10.1016/j.eswa.2022.117796>
- [11] Li X, Guo X, Tang H, et al (2022) Survey of integrated flexible job shop scheduling problems. *Computers & Industrial Engineering* 174:108786. <https://doi.org/10.1016/j.cie.2022.108786>
- [12] Roy B, Sussmann B (1964) Les problemes d’ordonnancement avec contraintes disjonctives. *Note DS* 9
- [13] Simonovsky M, Komodakis N (2017) Dynamic edge-conditioned filters in convolutional neural networks on graphs. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp 3693–3702, <https://doi.org/10.1109/CVPR.2017.11>
- [14] Song W, Chen X, Li Q, et al (2023) Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics* 19(2):1600–1610. <https://doi.org/10.1109/TII.2022.3189725>
- [15] Wan L, Fu L, Li C, et al (2024) Flexible job shop scheduling via deep reinforcement learning with meta-path-based heterogeneous graph neural network. *Knowledge-Based Systems* 296:111940. <https://doi.org/10.1016/j.knosys.2024.111940>
- [16] Zhang C, Song W, Cao Z, et al (2020) Learning to dispatch for job shop scheduling via deep reinforcement learning. In: *Advances in Neural Information Processing Systems*, pp 1621–1632
- [17] Zhang Z, Fu Y, Gao K, et al (2024) A cooperative evolutionary algorithm with simulated annealing for integrated scheduling of

distributed flexible job shops and distribution. *Swarm and Evolutionary Computation* 85:101467. <https://doi.org/10.1016/j.swevo.2023.101467>

## Statements and Declarations

### Funding

This work was supported by the Guangdong Basic and Applied Basic Research Foundation (2023A1515140071), the National Natural Science Foundation of China (U22A20106), and the Fundamental Research Funds for the Central Universities (FRF-TP-25-032).

### Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

### Ethics Approval and Consent

Not applicable.

### Author Contributions

All authors contributed to the study conception, methodology, implementation, analysis, and manuscript preparation. Yue Qi and Shihong Duan are the corresponding authors. All authors read and approved the final manuscript.

### Data and Code Availability

The datasets are available from the corresponding authors on reasonable request. The code is not publicly available because it forms part of ongoing research.